

Backend Architecture Design

Beere Kesava & Brothers Silks ERP

Production, Weaver, Inventory, Sales & Finance Management System

Prepared for: Engineering handoff — backend implementation

Source basis: Reverse-engineered from the existing React/TypeScript frontend (Figma Make prototype) — all Context files, page components, and design-note markdown files under `src/`

Current state: Pure client-side SPA, no backend/API/database exists yet. This document specifies the backend to build from scratch.

1. Executive Summary

Beere Kesava & Brothers Silks (Est. 1999) runs a saree weaving business spanning outsourced weavers, in-house factory looms, raw-material procurement, quality control, finishing, wholesale/retail sales, and multi-firm accounting. The current application is a fully client-side React + TypeScript SPA (Vite, MUI/Radix/Tailwind, React Router) with **no backend at all** — every entity (batches, orders, weavers, inventory, payments, purchase orders...) lives only in React Context state seeded with mock arrays, and auth is a fake localStorage flag with no password/OTP check.

This document defines the backend system needed to make the app real: a normalized relational schema, a REST API surface mapped 1:1 from the existing Context methods, an authentication & RBAC model for the 6 existing roles (plus finer-grained access levels), file storage for images/signatures/Excel uploads, and supporting services (notifications, audit log, reporting/analytics, barcode/QR generation).

1.1 Business Domains Identified

- **Procurement** — Vendors, raw-material Purchase Orders (Warp/Resham/Jari), GRN, Suppliers & external saree purchases, Purchase Requests with approval workflow
- **Production** — Batch creation, saree-row assignment to weavers or factory looms, Design Library, Material Issue (with signature capture), Factory Loom management
- **Quality & Finishing** — QC inspection with pay-deduction rules, Finishing assignment/return/dispatch, Quotations
- **Sales & Fulfilment** — Bulk (wholesale) Orders, unified saree/sales tracking, Inventory, Dispatch/Transport/Invoicing, Customers
- **Finance** — Weaver Payments (with Excel bulk-upload matching), Vendor Payments, Customer Invoices, 6 legal Firms each with independent income/expense ledgers, Outstanding & Rates/Pricing
- **Administration** — Users/staff with roles + access levels, Approvals queue, Audit Log, Notifications, Label/barcode settings, Reports

1.2 Roles (existing in frontend)

Role → Portal mapping

Field	Type / Notes
admin	Full operational admin portal (materials, production, orders, inventory, payments, etc.)
superadmin	Admin portal + Approvals queue (PO / purchase-request approval) + Audit Log
worker	Shop-floor staff portal: GRN, material issue, QC, finishing, dispatch, weaver liaison
weaver	Self-service portal: own batches, materials issued/received, payments, sarees
shop	Retail/shop staff portal
accountant	Payments-focused portal with restricted download/money-view access (see DownloadAccess/MoneyAcco

An additional per-user **AccessLevel** (e.g. Full Access vs restricted) layers on top of role, and an *admin-viewing-as-staff* mode exists for admins/superadmins to preview other portals.

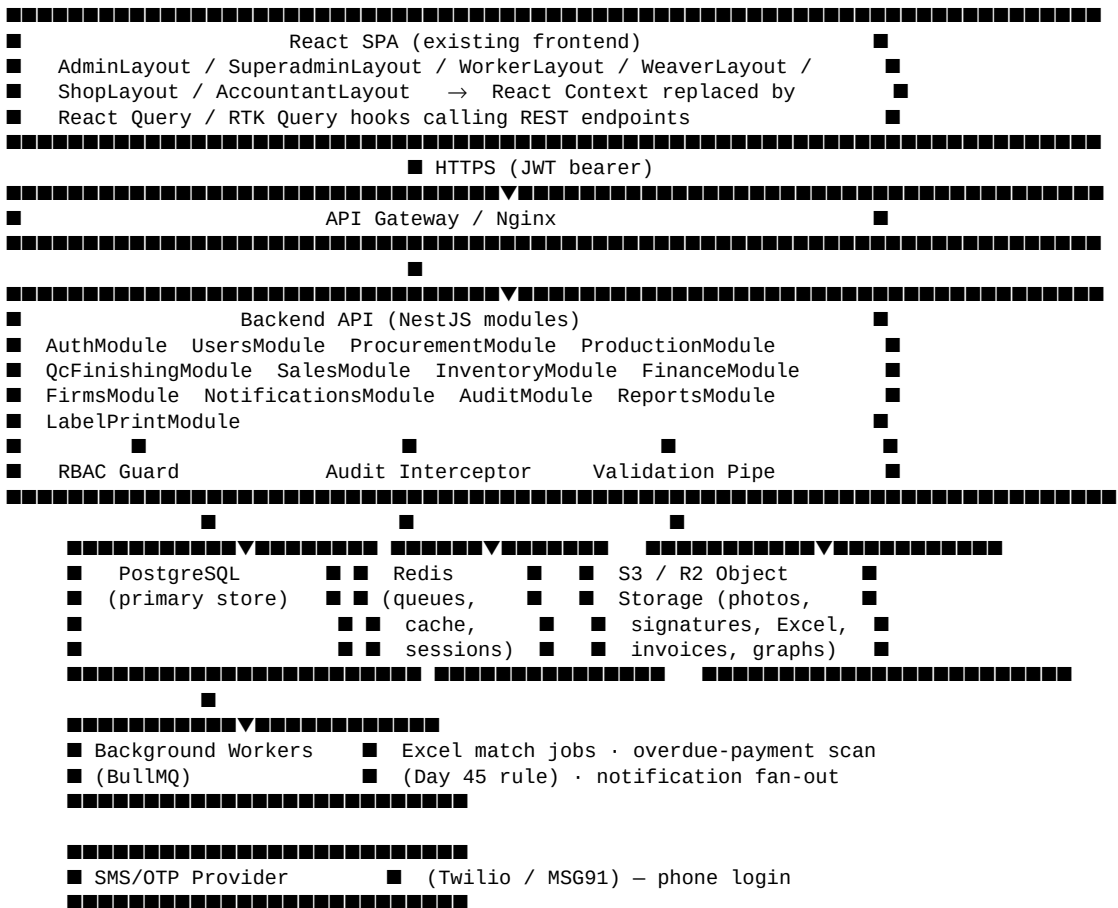
2. High-Level Architecture

Recommended stack (pragmatic, fits a small-to-mid business ERP with moderate concurrency, file uploads, and Excel-based reconciliation workflows):

Recommended technology stack

Field	Type / Notes
API Runtime	Node.js + TypeScript (shares types with the existing React/TS frontend) — Express or NestJS (NestJS re
Database	PostgreSQL 15+ (relational integrity matters heavily here: FK chains batch→saree→QC→finishing→disp
ORM	Prisma (typed schema matches the TS frontend models closely) or TypeORM
Auth	JWT access + refresh tokens; phone + OTP (SMS via MSG91/Twilio) matching the intended login-page-
File storage	S3-compatible object storage (AWS S3 / Cloudflare R2 / MinIO self-hosted) for: color-slip photos, design
Background jobs	BullMQ + Redis — for Excel-upload matching (weaver/vendor payments), notification fan-out, overdue-pa
Realtime	WebSocket (Socket.IO) or Server-Sent Events for the Notifications feed and live batch/QC status update
Barcode/QR	Server-side generation (bwip-js / qrcode npm libs) for saree tag printing, or client-side render + server-st
Reverse proxy / infra	Nginx or a managed gateway; Docker Compose for local dev, single VM or ECS/Fly.io for hosting given t

2.1 Logical Architecture Diagram



3. Authentication & Authorization

3.1 Login flow (replacing current fake auth)

- **POST /auth/otp/request** {phone} → sends OTP via SMS provider, rate-limited
- **POST /auth/otp/verify** {phone, otp} → returns JWT access token (short-lived, ~15min) + refresh token (httpOnly cookie, ~30 days) + user profile {id, empld, name, roles[], accessLevel}
- **POST /auth/select-role** {role} → for users with multiple assignable roles (mirrors existing RoleSelectPage); returns a role-scoped token/claim
- **POST /auth/refresh, POST /auth/logout** — logout also writes an AuditLog 'logout' event
- **Admin/Superadmin 'view as' mode:** POST /auth/impersonate-portal {role} issues a scoped, time-limited token flag (mirrors adminViewingAs) without changing the underlying identity — logged in AuditLog as impersonation

3.2 RBAC model

Two-layer permission model reflecting the frontend's Role + AccessLevel split:

Core RBAC tables

Field	Type / Notes
roles	admin, superadmin, worker, weaver, shop, accountant (seeded, extensible)
access_levels	Full Access / Restricted (view-only) / Download-Restricted / Money-Hidden — gates matching Download
permissions	Fine-grained action keys, e.g. po.approve, finance.view_amounts, export.download, weaver.pay
role_permissions / user_overrides	Default permission set per role, with per-user overrides for custom access levels

Every API route is guarded by a NestJS Guard checking (role $\subseteq$ allowed roles) AND (permission present). Money-related fields (payments, amounts, GST) are additionally filtered in the response serializer when accessLevel = Money-Hidden, to support the accountant portal's restricted views.

4. Core Data Model

Derived directly from the existing frontend Context/interface definitions. Types below map closely to Prisma/SQL table columns; JSON columns are used for embedded/variable-shape fields (materials list, photo URL arrays) where a separate join table isn't warranted.

users

Field	Type / Notes
id (PK)	uuid
emp_id	text unique, format EMP-NNN
first_name / last_name	text
mobile	text unique, +91 format
email	text nullable
role	enum: admin, superadmin, worker, weaver, shop, accountant
access_level	enum, see 3.2
portal	text (derived/display)
status	enum: active, inactive
date_added	timestamp

weavers

Field	Type / Notes
id (PK)	uuid
name, first_name, last_name	text (photo & email mandatory per erp-changelog.md)
initials	text, derived, used in saree ID e.g. RAVI-L2-001
village, cluster	text
looms	int — number of looms operated
status	enum: active, inactive
photo_url	S3 reference
email, phone	text
bank_name, account_no, ifsc	text — for payment matching

factory_looms

Field	Type / Notes
id (PK)	uuid
loom_number	text, e.g. BKB-F-0X
location	text
operator_name, operator_phone	text
status	enum: active, idle, maintenance
installed_year	int
notes	text

design_library

Field	Type / Notes
code (PK)	text unique
name, type_code, type_name	text
description, color	text
weaver_id (FK)	uuid nullable
notes_for_weaver	text
color_slip_photo_url, design_graph_url	S3 reference

saree_type_rates

Field	Type / Notes
code (PK)	text
type, description	text
making_charge, retail_price, wholesale_price	decimal
std_weight_g, warp_weight_g, resham_weight_g, digital_jari_weight_g	decimal, jari_weight_g derivable as reels via 230g/reel constant

batches

Field	Type / Notes
id (PK) — batch_id	text, business key
total_count	int
due_date	date
status	enum: draft, active, completed
created_at, updated_at	timestamp

batch_saree_rows (child of batches)

Field	Type / Notes
id (PK)	uuid
batch_id (FK)	text
serial	int, sequence within batch
saree_id	text nullable, unique once assigned — format {WeaverInitials}-L{loom}-{seq3} or BKB-L{loom}-{seq3}
recipient_type	enum: weaver, factory_loom
weaver_id / factory_loom_id (FK)	uuid nullable, exactly one set
design_code (FK), saree_type_code (FK)	text
bulk_order_ref (FK)	text nullable
qc_passed	boolean nullable

material_issue_records

Field	Type / Notes
id (PK)	text, format MIR-2026-NNN
weaver_id / factory_loom_id (FK)	uuid nullable
loom_number, batch_id (FK)	text nullable
issued_by (FK users), issued_at	uuid, timestamp

signature_method	enum: here, remote
signature_captured, signature_timestamp	boolean, timestamp
status	enum: pending_signature, signed, cancelled
notes	text

material_issue_items (child)

Field	Type / Notes
id (PK), issue_id (FK)	uuid
material_type	enum: Warp, Resham, Jari
warp_subtype	enum nullable: Resham Warp, Jari Warp
quantity, unit	decimal, text
jari_type, jari_grade (1G-5G), jari_color	enum/text nullable
grn_batch_id (FK)	text — links to GRN/PO receipt

qc_records

Field	Type / Notes
saree_id (PK, FK batch_saree_rows)	text
weaver_id / factory_loom_id (FK)	uuid nullable
batch_id, loom_number	text
result	enum: passed, semi, defective
defects	text[] / JSON array
making_charge, deduction, payable	decimal — semi: deduction ≤ making_charge; defective: payable = 0
received_date, qc_date	date
photo_url, notes, inspected_by (FK)	text/uuid

finishing_assignments / finishing_returns

Field	Type / Notes
id (PK)	uuid
saree_id (FK), design_code, saree_type	text
finishing_staff_id (FK), assigned_by (FK), assigned_date	text/uuid
status	enum: awaiting_return, returned
condition (on return)	enum: perfect, damaged
damage_type, damage_severity (Minor/Moderate/Severe), damage_notes, damage_photo_url	text/enum
quotation_ref (FK)	text nullable

finishing_staff

Field	Type / Notes
id (PK), emp_id	uuid, text EMP-NNN
first_name, last_name, mobile, email	text
specialisation, notes	text
status	enum: Active, Inactive

quotations

Field	Type / Notes
id (PK), quotation_number, quotation_date	uuid/text
customer_id (FK), bulk_order_ref (FK)	uuid/text nullable
apply_gst, gst_pct, subtotal, grand_total	boolean/decimal
firm_id (FK)	uuid nullable — one of the 6 legal firms
status	enum: raised, in_finishing, partially_received, received, dispatched
raised_by (FK), created_at	uuid, timestamp

quotation_sarees (child)

Field	Type / Notes
quotation_id (FK), saree_id (FK)	uuid/text
finishing_status	enum: pending, in_finishing, received
price	decimal (per-saree, from prices map)

bulk_orders

Field	Type / Notes
ref (PK)	text, format ORD-2026-NNN
customer_id (FK)	uuid
due_date, created_date	date
status	enum: on_track, at_risk, overdue (computed from due_date/daysLeft)
saree_type_code (FK), design_code (FK)	text
total, done, shortage	int
dispatch_status	enum: pending, dispatched, invoiced
payment_status	enum: pending, partial, paid
amount_due, amount_paid	decimal
gst_code, address, phone, visiting_card_url, person_urls	text/boolean
tallied, tallied_by, tallied_date	boolean/uuid/date

dispatch_records

Field	Type / Notes
id (PK)	uuid
type	enum: shop, wholesale
saree_ids	text[] / join table dispatch_sarees
dispatch_date, lr_number, transport_company, vehicle_number, driver_name	date/text
customer_id (FK)	uuid nullable
invoice_number, invoice_date, price_per_saree, total_amount, gst_pct, grand_total	text/decimal
firm_id (FK), payment_due_date	uuid/date
bulk_order_ref (FK), quotation_ref (FK)	text nullable

inventory_records

Field	Type / Notes
id (PK) — saree_id	text, FK to underlying saree
status	enum: qc_passed, finishing_complete, dispatched, damaged_review_needed
raw_type	enum: ready_saree, return
bulk_order_ref (FK), batch_id (FK), quotation_ref (FK)	text ref (FK)

sarees (unified saree master — new, unifies UnifiedSaree)

Field	Type / Notes
id (PK)	text, canonical saree ID
origin	enum: weaver, factory_loom, external
weaver_id / factory_loom_id / purchase_id (FK)	text, exactly one set per origin
batch_id (FK), design_code (FK), saree_type_code (FK)	text
weight_g, cost_price	decimal
qc_date	date
status	enum: unsold, retail, wholesale, returned
age_days	computed, not stored

sales / returns (child of sarees)

Field	Type / Notes
sale_ref (PK), saree_id (FK)	text
channel	enum: retail, wholesale
date, customer_id (FK), amount	date/uuid/decimal
return_ref, reason, refund_amount, restocked	text/decimal/boolean — nullable

suppliers / vendors

Field	Type / Notes
id (PK), name, initials	uuid/text
contact_name, phone, whatsapp, city, state	text/address
gst_code, specialty/type, terms	text
bank_name, account_no	text nullable
status	enum: active, inactive, overdue
rating	int — suppliers: raw sarees; vendors: raw materials (kept as distinct tables/roles per existing app)

purchases (external saree) / purchase_orders (raw material)

Field	Type / Notes
purchases.id (PK), supplier_id (FK)	uuid
date, saree_count, gst_number, invoice_number, bill_number, partial	number, integer, string
purchase_orders.id (PK), po_number	text, format PO-2026-NNN
vendor_id (FK), delivery_date, total_value, currency	uuid, date, decimal, enum: pending, approved, rejected, received
grn_id	text — links PO receipt to material_issue_records.grn_batch_id

purchase_requests

Field	Type / Notes
id (PK), supplier_id (FK), requested_by (FK)	uuid
saree_type, quantity, estimated_amount, urgency, reason	text
status	enum: pending, approved, rejected
decided_by (FK), decided_date, decision_note	date/text nullable

firms (6 legal billing entities)

Field	Type / Notes
id (PK), firm_name	uuid/text
gst_number, address, account_number, ifsc_code, bank_name	text
contact_person_name, contact_person_phone	text

firm_financial_entries

Field	Type / Notes
id (PK), firm_id (FK)	uuid
kind	enum: income, expense, misc
category	enum — income: Wholesale Sale/Retail Sale/Other; expense: Weaver Payments/Material Purchase/Shop
description, amount, date, notes	mixed

weaver_payments

Field	Type / Notes
id (PK), weaver_id (FK)	uuid
amount_paid, utr_number, firm_id (FK), payment_text, uploaded_at	decimal/text/uuid
batch_no (FK), loom_number, no_of_sarees, no_of_rows, no_of_rows_matched	integer/integer/integer/integer/integer Excel-matched rows

invoices (customer)

Field	Type / Notes
id (PK), customer_id (FK)	uuid
invoice_date, due_date, total, paid, status	date/decimal/enum: Paid, Partial, Pending, Overdue

invoice_payments / vendor_payments (child tables)

Field	Type / Notes
amount, date, utr, method, firm_id (FK)	mixed — same shape both sides

customers

Field	Type / Notes
id (PK), name, city, phone, address	uuid/text
gst_code, type	text — enum: wholesale, retail
visiting_card_url	S3 reference

audit_log

Field	Type / Notes
id (PK), user_id (FK)	uuid
status	enum: login, logout, failed
device, duration, fail_reason	text nullable
created_at	timestamp

notifications

Field	Type / Notes
id (PK), user_id/role (FK/enum) target	uuid/enum
type, payload (JSON), read_at, created_at	mixed — fan-out from domain events: PO approved, QC defect, payment overdue (Day 45), etc.

5. REST API Surface (by module)

Each existing Context mutator (e.g. addBulkOrder, approvePO, recordSale) maps to one endpoint below. All endpoints are versioned under /api/v1 and require a bearer JWT + RBAC guard unless noted.

Auth

Method & Path	Purpose
POST /auth/otp/request	send OTP to phone
POST /auth/otp/verify	verify OTP, issue tokens
POST /auth/select-role	choose active role for session
POST /auth/refresh · POST /auth/logout	token refresh / logout (+ audit log)

Users & Staff

Method & Path	Purpose
GET/POST /users	list / create staff (AddUserPage)
PATCH /users/:id, PATCH /users/:id/access-level	update profile / access level
GET /audit-log	superadmin audit trail

Procurement

Method & Path	Purpose
GET/POST /vendors, GET/POST /suppliers	master data
GET/POST /purchase-orders, POST /purchase-orders/material-approval-flow	create material PO, approval workflow
POST /purchase-orders/:id/grn	goods receipt against PO
GET/POST /purchases	external saree purchases
GET/POST /purchase-requests, POST /purchase-requests/supplier-purchase-request	supplier purchase request, approval queue
GET /approvals	superadmin unified approval queue (POs + requests)

Production

Method & Path	Purpose
GET/POST /design-library, POST /design-library/master-dispatch	design master dispatch
GET/POST /batches, PATCH /batches/:id/rows/batch-creation	batch creation, assign saree row to weaver/loom
GET/POST /factory-loom, GET/POST /factory-loom/batch	in-house loom management
POST /material-issues (+ signature capture), GET /material-issues	issue raw material
GET /weavers/:id/material-summary	issued/received/outstanding grams, reels

QC & Finishing

Method & Path	Purpose
POST /qc/:sareeId, GET /qc-history	record QC result, pay/deduction computed
POST /finishing/assignments, POST /finishing/sign-off	assign to finishing staff, record return/damage
GET/POST /quotations, POST /quotations/:id/quotation-approve	quotation management
GET/POST /finishing-staff	finishing staff roster

Sales & Fulfilment

Method & Path	Purpose
GET/POST /bulk-orders, PATCH /bulk-orders/:id	make/sale order lifecycle
GET/POST /customers	customer master
GET /inventory, GET /stock	unified stock views
POST /dispatch (shop wholesale), GET /dispatch/:id	dispatch/transport+ invoice generation
GET /sarees/:id, GET /sarees?filter=...	unified saree/sale/return tracking (SalesContext replacement)

Finance

Method & Path	Purpose
GET/POST /weaver-payments, POST /weaver-payments/:id	weaver payment (E-commerce)
GET/POST /vendor-payments, POST /vendor-payments/:id	vendor payment (E-commerce)
GET/POST /invoices, POST /invoices/:id/payments	customer invoicing
GET/POST /firms, GET/POST /firms/:id/ledgers	6-firm income/expense/misc ledgers
GET /rates-pricing, PATCH /rates-pricing/:id	sale type rate card
GET /outstanding	aggregated outstanding stock/sold/produced report

Platform

Method & Path	Purpose
GET /notifications, PATCH /notifications/:id	infinite notification feed
GET /reports/:type	dashboard/report aggregations
GET/POST /label-settings, POST /labels/:sareeId	barcode/QR tag generation
POST /scan/lookup	mobile barcode/QR scan → saree lookup (MobileScanView)

6. Cross-Cutting Concerns

6.1 ID generation

- Batch: business-defined batch_id (see existing seeds)
- Saree: {WeaverInitials}-L{loom}-{seq3} (weaver) or BKB-L{loom}-{seq3} / BKB-F-0X-{seq3} (factory)
- Purchase Order: PO-{year}-{seq3}; Bulk Order: ORD-{year}-{seq3}; Material Issue: MIR-{year}-{seq3}; Employee: EMP-{seq3}
- All sequence counters should be DB-side (Postgres sequences or a per-prefix counter table with row locking) to avoid client-side collision, unlike the current frontend which invents IDs in memory.

6.2 Business rules to enforce server-side

- QC pay rule: defective → payable = 0; semi → deduction capped at making_charge; passed → payable = making_charge. Enforce as a DB check constraint + service-layer validation, not just UI logic.
- Jari inventory always convertible/displayed in both Buns and Reels: 1 Bun = 4 Reels = 230g/reel — store canonical grams, compute both display units.
- Every saree must be tagged with a production source (weaver loom vs in-house factory loom) — enforce via a DB CHECK that exactly one of weaver_id/factory_loom_id is set on sarees/batch_saree_rows.
- Intended flow QC Passed → Stock → Sale Confirmed → Finishing → Dispatch (per [erp-changelog.md](#)) should be modeled as an explicit state machine on the saree/inventory status field, with guarded transitions, rather than left implicit as in the current frontend.
- Bulk-order payment overdue alert fires at Day 45 post-dispatch (not 60/90) — implement as a scheduled job (BullMQ repeatable) scanning `dispatch_records.payment_due_date`.

6.3 File uploads

All image/document fields (color slip photos, design graphs, signature captures, damage photos, visiting cards, Excel files, invoice PDFs) upload via a pre-signed S3 URL flow: client requests a signed PUT URL from `POST /uploads/sign`, uploads directly to storage, then submits the resulting object key with the parent entity's create/update request. Keeps large binaries off the API servers.

6.4 Excel import/matching (weaver & vendor payments)

Existing `PaymentsPage.tsx` implements client-side Excel parsing + fuzzy matching against weaver/vendor records, producing Matched/Unmatched result sets. Move this to a backend job: `POST /weaver-payments/import` uploads the file, enqueues a BullMQ job that parses rows (xlsx/exceljs), matches against weaver name/UTR/loom, and returns a job id; client polls `GET /jobs/:id` for the MatchedPayment/UnmatchedRow result, then confirms via `POST /weaver-payments/import/:jobId/confirm`.

6.5 Auditing & observability

- Every login/logout/failed-login → audit_log row (already modeled in frontend, just needs a real backend writer)
- Every approval/rejection (PO, purchase request) → audit_log with actor + reason
- Structured logging (pino) + request tracing; Sentry for error tracking
- Rate-limit OTP request/verify endpoints to prevent SMS abuse

7. Migration Plan (from prototype to real backend)

- **Phase 1 — Foundations:** stand up Postgres schema (section 4), NestJS project skeleton, JWT/OTP auth, S3 bucket, seed roles/firms/saree-type-rates from existing frontend mock data.
- **Phase 2 — Master data & procurement:** Users, Weavers, Vendors, Suppliers, Design Library, Purchase Orders + approvals, Purchase Requests.
- **Phase 3 — Production:** Batches, Factory Looms, Material Issue (with signature upload), replacing BatchContext/MaterialIssueContext/FactoryLoomPage in-memory logic.
- **Phase 4 — QC & Finishing:** QC records with pay-rule enforcement, Finishing assignments/returns, Quotations.
- **Phase 5 — Sales & Inventory:** Bulk Orders, Customers, unified Sarees/Inventory table (replacing the pseudo-random SalesContext.buildDataset()), Dispatch + invoicing.
- **Phase 6 — Finance:** Weaver/Vendor payments incl. async Excel-matching jobs, Customer Invoices, Firm ledgers, Rates/Pricing, Outstanding report.
- **Phase 7 — Platform:** Notifications (websocket), Audit Log, Reports/analytics aggregation, Label/barcode printing, Mobile scan lookup.
- **Phase 8 — Frontend cutover:** replace each React Context provider's in-memory reducer with React Query hooks calling the new REST API 1:1, remove localStorage-based auth, keep component/page structure unchanged.

Recommendation: build backend module-by-module in the phase order above and swap the corresponding frontend Context for API calls incrementally, so the app stays runnable end-to-end throughout the migration rather than requiring a big-bang rewrite.

*Document generated from a full source-tree analysis of the beere-kesava frontend repository, including all *Context.tsx data models, page components, and design-intent markdown notes under src/imports/pasted_text/.*